Game Client Integration

Introduction

This document describes integration between *RGS* (*Remote Game Server*) and *Game Client*.

The key words `MUST`, `MUST NOT`, `REQUIRED`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `RECOMMENDED`, `MAY`, and `OPTIONAL` in this document are to be interpreted as described in RFC 2119.

Connector

There is separate frontend library called Connector which simplifies this integration. Consider using the library instead of integrating the API directly

Integration

Prerequisites:

- Communication `SHOULD` be done via `HTTP/1.1` and encrypted with `SSL/TLS`.
- All API requests `MUST` be passed with `Content-Type: application/json`.
- All API responses `MUST` be passed with `Content-Type: application/json`.
- Any additional response or request data, parameters, fields or codes not described in documentation `SHOULD NOT` be used.
- Character encoding utf-8 `SHOULD` be set to `UTF-8`
- All time and date fields `SHOULD` be in `UTC` timezone

Authentication

Token authentication (also called bearer authentication) is an HTTP authentication scheme that involves security tokens called bearer tokens. The bearer token is a cryptic string, usually generated by the server in response to a login request. The client `MUST` send this token in the Authorization header when making requests to protected resources:

`Authorization: Bearer <token>`

Response codes

All responses return appropriate response's codes.

Code	Status
`2XX`	Successful
`4XX`	Application Error
`5XX`	Server Error

Errors

All errors follow unified structure.

Field `message` on non-production environments gives more verbose description of the problem to help troubleshooting. On production environment it is replaced with very generic statement for security reasons.

Field `code` is the same across all environments and should be used to display localised message to the end user. Available codes: `APPLICATION_ERROR`, `SERVER_ERROR`, `PLAYER_UNAUTHORIZED`, `SERVER_UNAUTHORIZED`, `INSUFFICIENT_FUNDS`, `LOSS_LIMIT`, `UNKNOWN`.

```
{
    "error": {
        "message": "Error description",
        "code": "APPLICATION_ERROR"
    }
}
```

API

Path: `/authenticate` **Method:** `POST` **Authorization:** `NO`

Authenticate player and start a new session. Slotify comes with built-in `demo` *Operator* which will sucessfully authenticate using any `key`.

Body

Name	Type		Example	Description
`key`	`string`	`REQUIRED`	`dnsa89me329jdos`	*Operator*'s session initialisation key.
`operator`	`string`	`REQUIRED`	`myOperator`	*Operator*'s id

Response

Name	Type		Example	Description
`balance`	`number`	`REQUIRED`	`123.45`	Balance in *Player*'s currency
`currency`	`string`	`REQUIRED`	`eur`	*Player*'s currency (internal platform code)
`currencySymbol`	`string`	`REQUIRED`	`eur`	*Player*'s currency (to display to the player)
`currencyDecimals`	`number`	`REQUIRED`	`2`	*Player*'s currency decimal places
`jurisdiction`	`string`	`OPTIONAL`	`mt`	*Player*'s jurisdiction
`playerId`	`string`	`REQUIRED`	UUID	*Player*'s id on the rgs
`nickname`	`string`	`OPTIONAL`	`my nickname`	*Player*'s nickname

Path: `/game/play` **Method:** `POST` **Authorization:** `YES`

Creates a new wager.

Body

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`provider`	`string`	`REQUIRED`	`myProvider`	Provider Id

Name	Type		Example	Description
`action`	`string`	`REQUIRED`	`gamble`	Action
`bet`	`number`	`REQUIRED`	`2.58`	Cash bet in player's currency. It `SHOULD NOT` have more then 2 decimal places, otherwise it will be rounded using bankers rounding.
`sideBets`	`object`	`OPTIONAL`	`{"double": 5, "triple": 10}`	Side bets
`roundId`	`string`	`OPTIONAL`	`123e4567-e89b-12d3-a456-426614174000`	Round Id for continuation
`cheat`	`string`	`OPTIONAL`	`bonus`	Cheat for forcing game outcome. Available only in `development` mode

Response

Name	Type		Example	Description
`wager`	`object`	`REQUIRED`		See Wager
`roundId`	`string`	`REQUIRED`	`123e4567-e89b-12d3-a456-426614174000`	Round Id used for multi-step rounds
`balance`	`number`	`OPTIONAL`	`200.56`	Player's current balance in player's currency, sent only if changed

---

Path: `/game/complete` **Method:** `POST` **Authorization:** `YES`

Completes all transactions and transfers money to player's account.

Body

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`provider`	`string`	`REQUIRED`	`myProvider`	Provider Id
`roundId`	`string`	`OPTIONAL`	`123e4567-e89b-12d3-a456-426614174000`	Round Id for continuation

Response

Name	Type		Example	Description
`balance`	`number`	`OPTIONAL`	`200.56`	Player's current balance in player's currency, sent only if changed

---

Path: `/game/info` **Method:** `GET` **Authorization:** `YES`

Returns configuration and other information

Query Parameters

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`provider`	`string`	`REQUIRED`	`myProvider`	Provider Id

Response

Name	Type		Example	Description
`config`	`any`	`OPTIONAL`	`{"paytable": {...}}`	Game config
`bets`	`any`	`OPTIONAL`	`{"main": {...}, "gamble": {...}}`	Bet config per action. See Bet Config
`state`	`any`	`OPTIONAL`	`{"collection": 10}`	Player's game state

Path: `/game/recover` **Method:** `POST` **Authorization:** `YES`

Returns unfinished rounds

Body

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`provider`	`string`	`REQUIRED`	`myProvider`	Provider Id

Response

Name	Type		Example	Description
`rounds`	`Round[]`	`OPTIONAL`		List of rounds See Round

Path: `/game/feed` **Method:** `GET` **Authorization:** `YES`

Returns feed stored by the game

Query Parameters

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`amount`	`number`	`REQUIRED`	`4`	Amount of wager entries to retrieve

Response

Name	Type		Example	Description
`feed`	`any[]`	`OPTIONAL`	`[1, 2, 26, 36]`	List of wager entries from the feed

Path: `/game/cheats` **Method:** `GET` **Authorization:** `NO`

Lists available cheats. Available only in `development` mode.

Query Parameters

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`provider`	`string`	`REQUIRED`	`myProvider`	Provider Id

Response

Name	Type		Example	Description
`cheats`	`string[]`	`REQUIRED`	`["bonus", "any win"]`	Cheats

Path: `/game/replay` **Method:** `GET` **Authorization:** `YES`

Returns round

Query Parameters

Name	Type		Example	Description
`game`	`string`	`REQUIRED`	`mySlot`	Game Id
`provider`	`string`	`REQUIRED`	`myProvider`	Provider Id
`roundId`	`string`	`REQUIRED`	`db76b227-0582-45bc-a2cd-fbf38449f28e`	Round Id

Response

Name	Type		Example	Description
`round`	`Round`	`OPTIONAL`		See Round
`currency`	`string`	`REQUIRED`	`eur`	Player's currency
`config`	`any`	`OPTIONAL`	`{"paytable": {...}}`	Game config
`bets`	`any`	`OPTIONAL`	`{"main": {...}, "gamble": {...}}`	Bet config per action. See Bet Config
`state`	`any`	`OPTIONAL`	`{"collection": 10}`	Player's game state

Path: `/currencyDecimals` **Method:** `GET` **Authorization:** `NO`

Lists number of decimal digits for each currency configured on the RGS.

Response

Name	Type		Example	Description
N/A	`{[key:string]: number}`	`REQUIRED`	`{"btc": 8}`	Decimals for each currency

Path: `/proveFairness` **Method:** `GET` **Authorization:** `NO`

Endpoint supports both single-player and multiplayer provable fairness. Returns proof of the fair gameplay for a given `seeds` and `nonce`, or given `seed` and `hash` in multiplayer case. Game server might also require additional information for precise proof generation.

Query Parameters

Name	Type		Description
`rngState`	`RoundRngState` or `DrawRngState`	REQUIRED	server seed
`data`	`any`	OPTIONAL	data specific for the game, to allow precise proof

Returns

Name	Type		Example	Description
.	`FairnessProof`	`REQUIRED`		See FairnessProof

Types

Round

Name	Type		Example	Description
`roundId`	`string`	`REQUIRED`	`db76b227-0582-45bc-a2cd-fbf38449f28e`	Round Id
`wagers`	`Wager[]`	`REQUIRED`		List of wagers. See Wager

Wager

Name	Type		Example	Description
`data`	`any`	`OPTIONAL`	`[{"spin": 1}, {"spin": 2}]`	Game outcome
`state`	`any`	`OPTIONAL`	`{"collection": 10}`	Player's game state persistent between rounds

Name	Type		Example	Description
`win`	`number`	`REQUIRED`	`16.45`	Cash win in Player's currency
`sideWins`	`object`	`OPTIONAL`	`{"double": 5, "triple": 10}`	Wins from side bets
`next`	`string[]`	`OPTIONAL`	`["gamble", "take"]`	Next actions for multi-wager rounds. If response contains `next` array then in next play request `action` should be set to one of the array's elements.

Bet Config

Name	Type		Example	Description
`available`	`number[]`	`REQUIRED`	`[0.01, 0.5, 1, 5, 10]`	Available bets in Player's currency
`default`	`number`	`REQUIRED`	`0.5`	Default bet in Player's currency
`coin`	`number`	`REQUIRED`	`25`	Coin

RoundRngState

Name	Type	Example	Description
`serverSeed`	`string`	`69742f105765e8d0a35a5819918a8d945f2a8f9f211e60c308a83c2af3a2c759`	server seed
`clientSeed`	`string`	`my-client-seed`	client seed
`nonce`	`number`	`13`	nonce

DrawRngState

Name	Type	Example	Description
`hash`	`string`	`69742f105765e8d0a35a5819918a8d945f2a8f9f211e60c308a83c2af3a2c759`	server hash
`seed`	`string`	`players-seed`	players' seed

FairnessProof

Name	Type		Example	Description
`hashes`	`Hash[]`	`REQUIRED`		List of hashes generated over the course of the given round, to extract random numbers. See Hash
`randomizations`	`Randomization[]`	`REQUIRED`		List of randomizations requested by the game for a given round. See Randomization

Hash

Name	Type		Example	De
`hex`	`string`	`REQUIRED`	`634ce936de372e6033083cbce378edaa53734ac19f69ebc193cf44fe5dcb29fc`	He rep 25(has
`bytes`	`number[]`	`REQUIRED`	`[99, 76, 233, 54, 222, 55, 46, 96, 51, 8, 60, 188, 227, 120, 237...]`	Arr byt rep 25(has

Randomization

Name	Type		Example	Description
`limit`	`number`	`REQUIRED`	`10001`	Limit requested by the game for a given randomization, i.e. `random(limit)` call made by the game
`extractions`	`Extraction[]`	`REQUIRED`		A list of attempts to get the integer satisfying limit condition, as in Unbiased Integer Randomisation Algorithm used in the system. See Extraction
`randomNumber`	`number`	`REQUIRED`	`2753`	Resulting random number
`gameEvent`	`any`	`REQUIRED`	`{roll: 27.53}`	Game specific result that is generated based on the current randomization

Extraction

Name	Type		Example	Description
`cursor`	`number`	`REQUIRED`	`4`	Cursor value for a given extraction
`hashIndex`	`number`	`REQUIRED`	`0`	Equal to `Math.floor(cursor / 8)` - index of a hash from which the current extraction is performed; refers to the `hashes` list
`offset`	`number`	`REQUIRED`	`4`	Equal to `cursor % 8` - offset within a current hash, which points to the integer being extracted
`integer`	`number`	`REQUIRED`	`1400064705`	Integer returned by the extraction for Unbiased Integer Randomization algorithm. This integer might still be rejected by the algorithm, resulting in next extraction attempt